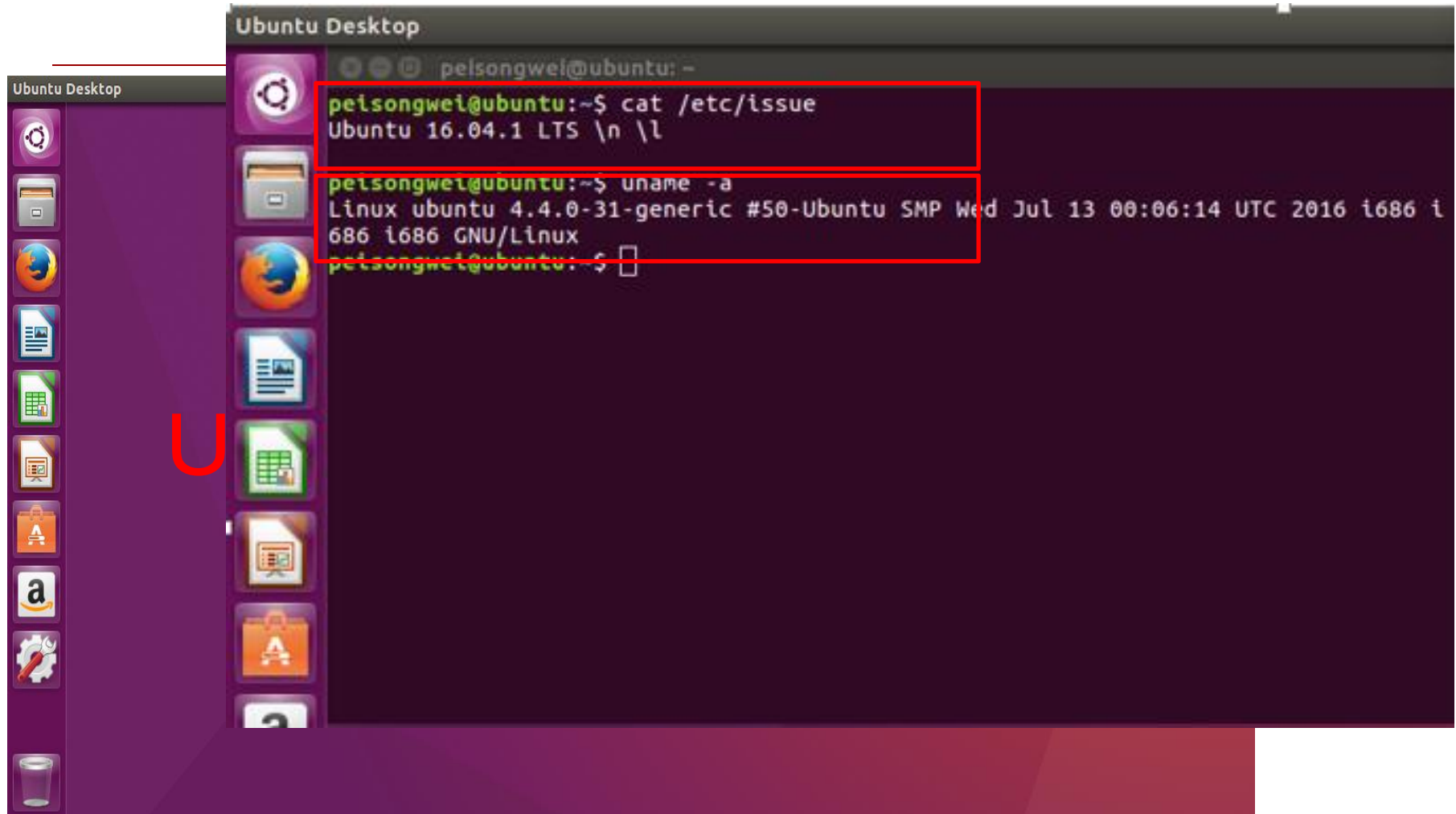




实验四

字符型I/O设备驱动

Linux操作系统



Linux内核

- Linux内核是Linux操作系统不可缺少的组成部分，但是内核本身不是操作系统。许多Linux操作系统发行商如RedHat、Debian等都采用Linux内核，然后加入用户需要的工具软件和程序库，最终构成一个完整的操作系统。嵌入式Linux系统是运行在嵌入式硬件系统上的Linux操作系统，每个嵌入式Linux系统都包括了必要的工具软件和程序库。

Linux内核

- 内核是操作系统的核心部分，为应用程序提供安全访问硬件资源的功能。直接操作计算机硬件是很复杂的，内核通过硬件抽象的方法屏蔽了硬件的复杂性和多样性。通过硬件抽象的方法，内核向应用程序提供了统一和简洁的接口，应用程序设计复杂程度降低。实际上，内核可以被看做是一个系统资源管理器，内核管理计算机系统中所有的软件和硬件资源。

Linux内核

- 应用程序可以直接运行在计算机硬件上而无需内核的支持，从这个角度看，内核不是必要的。在早期的计算机系统中，由于系统资源的局限，通常采用直接在硬件上运行应用程序的办法。运行应用程序需要一些辅助程序，如程序加载器、调试器等。随着计算机性能的不断提高，硬件和软件源都变得复杂，需要一个统一管理的程序，操作系统的概念也逐渐建立起来。

Linux内核版本

- Linux内核版本号采用两个“.”分割的三个数字来标示，形式为“X.Y.Z”。其中，X是主要版本号，Y是次要版本号，Z代表补丁版本号。奇数代表不稳定的版本；偶数代表稳定的版本。“稳定”和“不稳定”是相对的，如Linux内核1.1.0相对于1.0.0来说是“不稳定”版本，但是与1.1.1对比是“稳定”版本。在Linux内核开发过程中，“不稳定”版本通常是在原有版本基础上增加了新的功能或者新的特性。

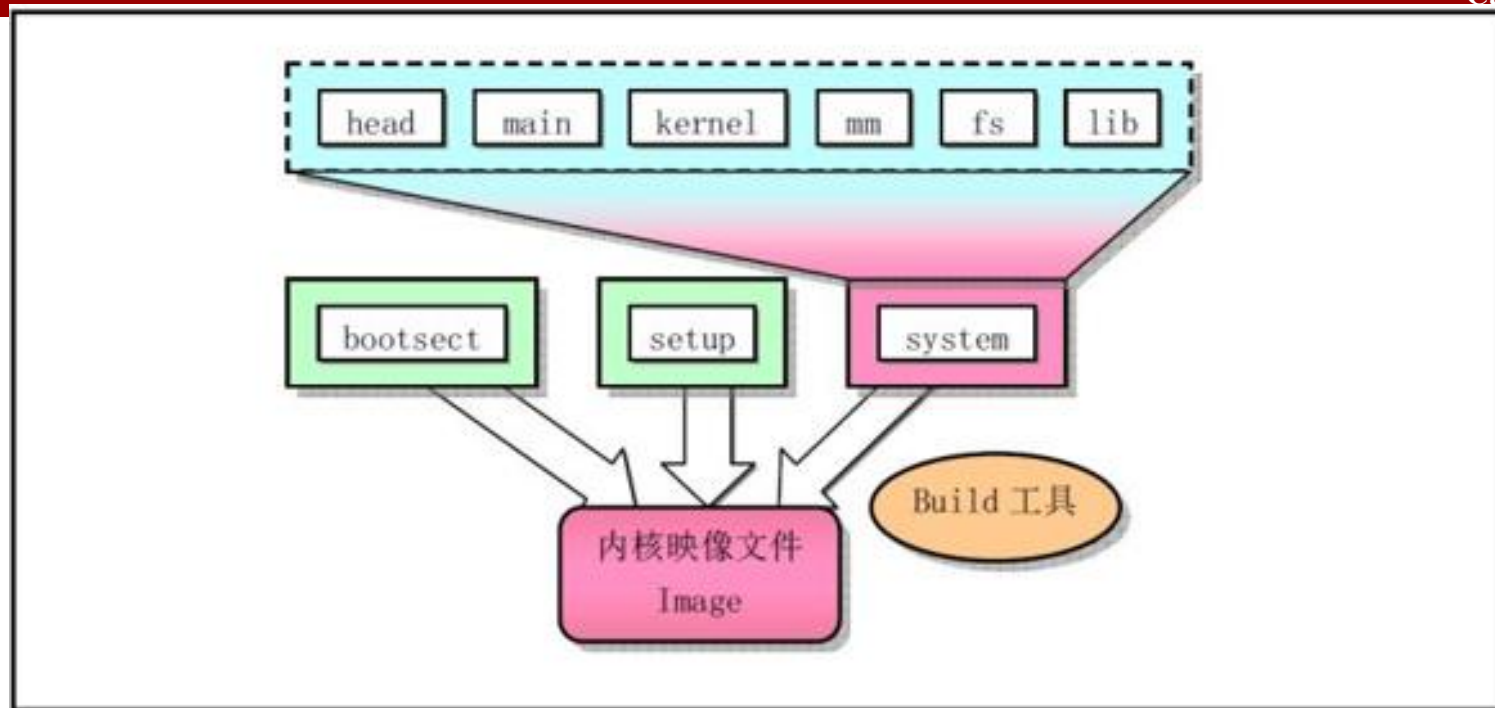
实验采用Linux内核版本Linux-0.11?

- 由于最新Linux内核源代码非常庞大。Linux-0.11与目前Linux内核基本功能较为接近，又非常短小的0.11版本作为入门学习的最佳版本。0.11内核源代码只有一万四千行左右，其中包括的内容基本上都是linux系统的精髓。

使用tar命令将linux-0.11.tar.gz解压时，结构如下

linux	
— boot	系统引导汇编程序
— fs	文件系统
— include	头文件(*.h)
— asm	与 CPU 体系结构相关的部分
— linux	Linux 内核专用部分
— sys	系统数据结构部分
— init	内核初始化程序
— kernel	内核进程调度、信号处理、系统调用等程序
— blk_drv	块设备驱动程序
— chr_drv	字符设备驱动程序
— math	数学协处理器仿真处理程序
— lib	内核库函数
— mm	内存管理程序
— tools	生成内核 Image 文件的工具程序

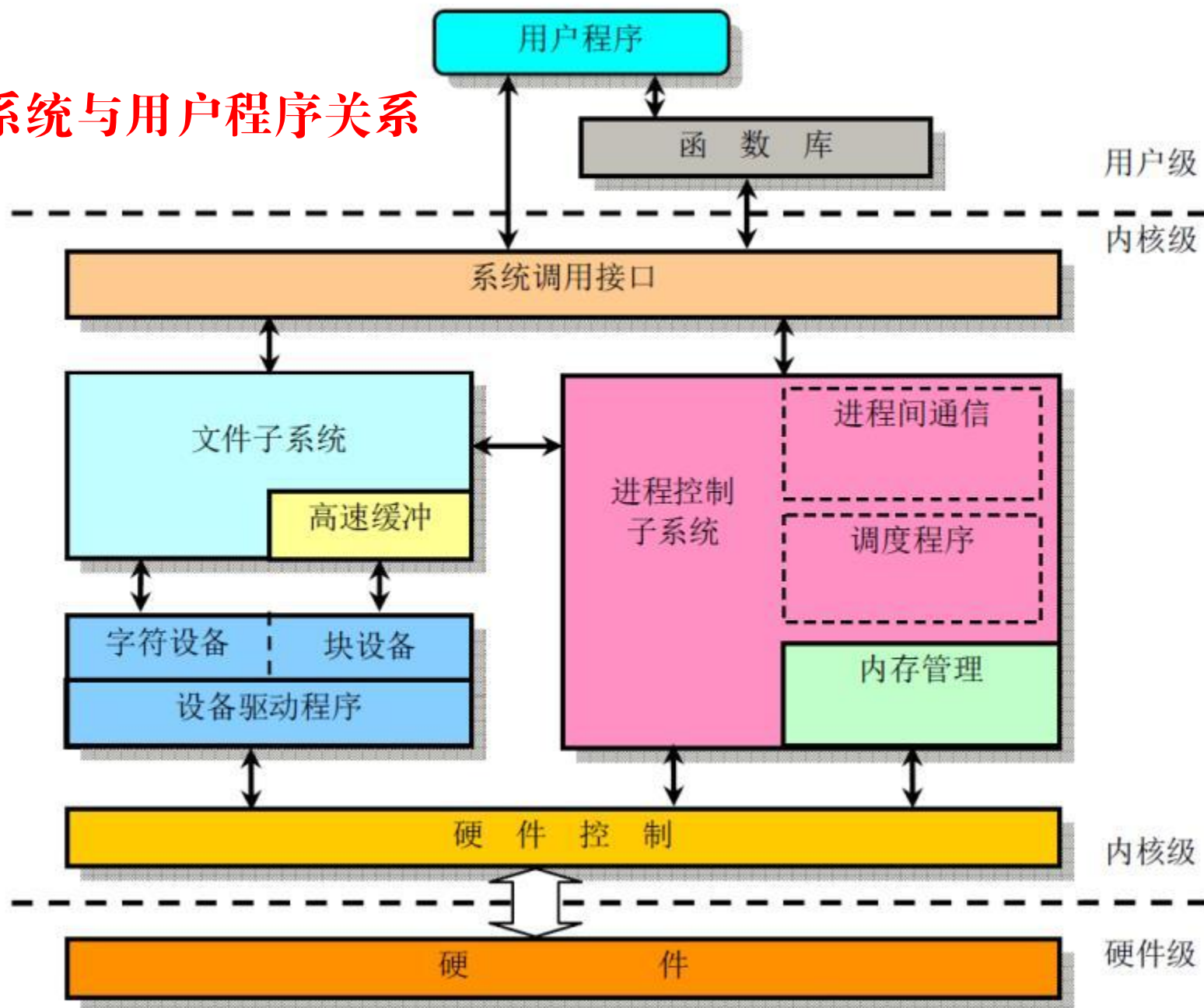
Linux内核源代码目录结构



内核编译连接组合结构

- 具体来讲，是对boot/中的`bootsect.s`、`setup.s`使用8086汇编器进行编译，分别生成各自的执行模块。再对源代码中的其他所有程序使用GNU的编译器`gcc/gas`进行编译，并连接成模块`system`。再用build工具将这三块组合成一个可运行的内核映像文件`image`。

内核系统与用户程序关系



内核空间与用户空间

- Linux的两种运行模式：
 - 内核模式
 - 用户模式
- 内核模式对应内核空间，而用户模式对应用户空间。
- 驱动程序作为内核的一部分，它对应内核空间，应用程序不能直接访问其数据

❖ 区分用户态和内核态目的在于安全考虑：

➤ 禁止用户程序和底层硬件直接打交道

(最简单的例子，如果用户程序往硬件控制寄存器写入不恰当的值，可能导致硬件无法正常工作)

➤ 禁止用户程序访问任意的物理内存

(否则可能会破坏其他程序的正常执行，如果对内核所在的地址空间写入数据的话，会导致系统崩溃)

用户程序如何同设备打交道？

例如，用户需通过网卡发送数据

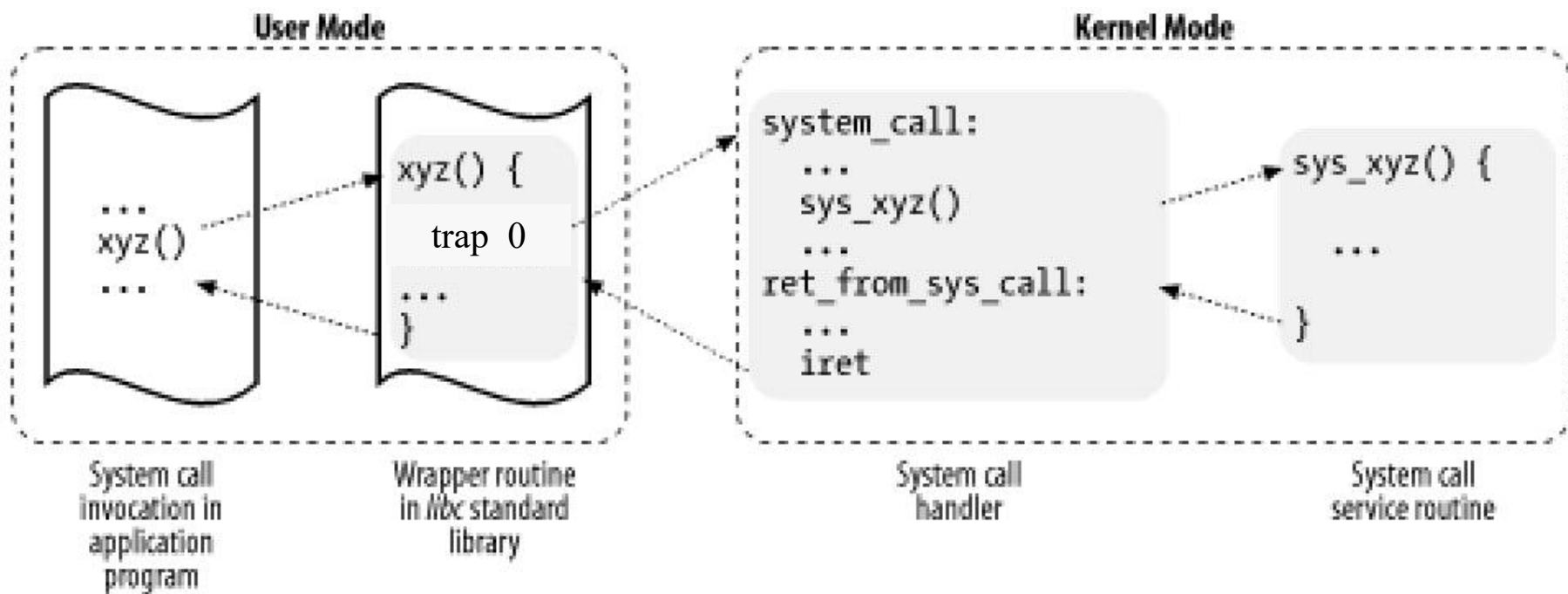
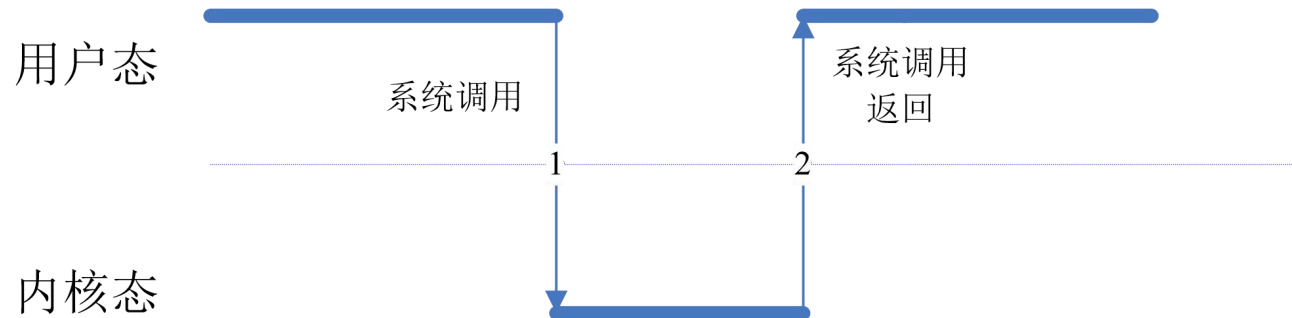
- 硬件被linux 内核隔离，只能通过内核实现。
- 不可能直接调用操作系统的函数：不可行，也不安全。

➤ Linux提供的解决方法：系统调用

系统调用的意义

- 操作系统为用户态进程与硬件设备进行交互提供了一组接口——系统调用
 - 把用户从底层的硬件编程中解放出来
 - 极大的提高了系统的安全性
 - 使用户程序具有可移植性
- Linux系统中，系统调用接口，即中断调用
`int 0x80`

● 系统调用图解

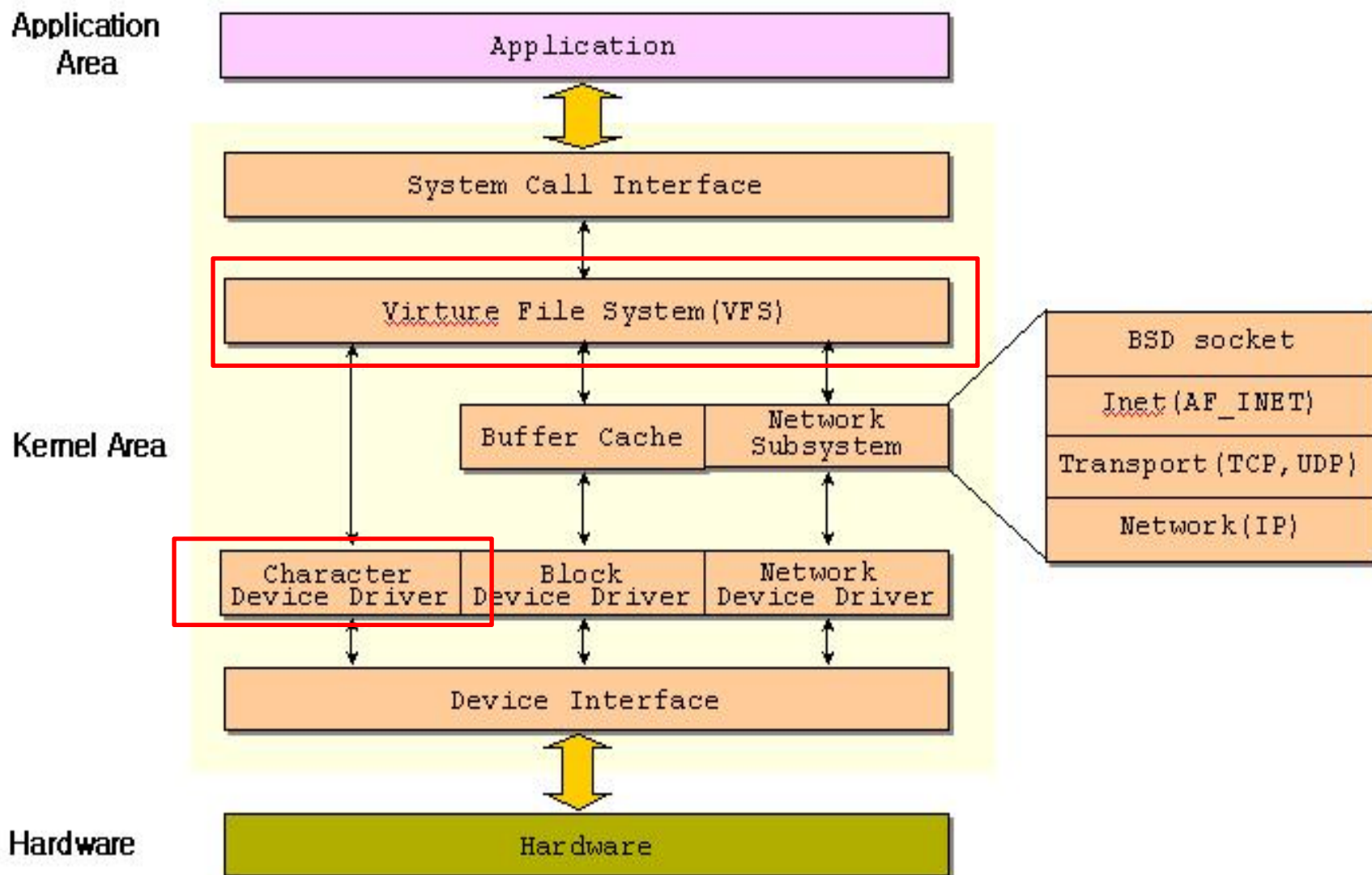


- Linux 2.6 提供了300多个系统调用，用户可以通过这些系统调用，及它们的组合实现对设备的操作。
- 通常，应用程序开发并不直接和系统调用打交道，而是用C库提供的一层包装函数。

如，`malloc()` → `sbrk()` → `sys_brk` → 内核函数
`sys_brk` 是45号系统调用，C库中它的系统调用方式可能是：

```
...  
movi  r1, 45  
trap  
...
```


内核中设备驱动的层次

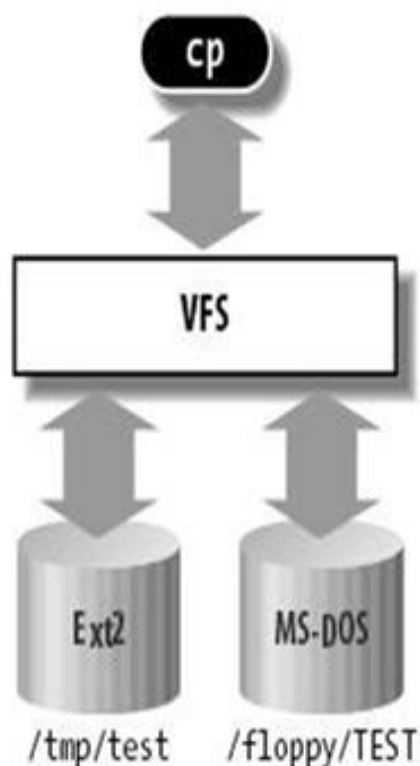


虚拟文件系统 Virtual Filesystem Switch

- VFS是一个软件层，是用户应用程序与具体文件系统实现之间的抽象层：
 - 对用户界面：一组标准的、抽象的文件操作，以系统调用提供，如`read()`、`write()`、`open()`等。
 - 对具体文件系统界面：主体是`file_operations`结构，全是函数指针，提供函数跳转表。

VFS在一个简单文件复制操作中的作用

- 假设用户输入shell命令：\$ cp /floppy/TEST /tmp/test
 - 在cp命令中，它通过VFS提供的系统调用接口进行文件操作。



```
inf = open("/floppy/TEST", O_RDONLY, 0);
outf = open("/tmp/test",
            O_WRONLY|O_CREAT|O_TRUNC, 0600);
do {
    i = read(inf, buf, 4096);
    write(outf, buf, i);
} while (i);
close(outf);
close(inf);
```

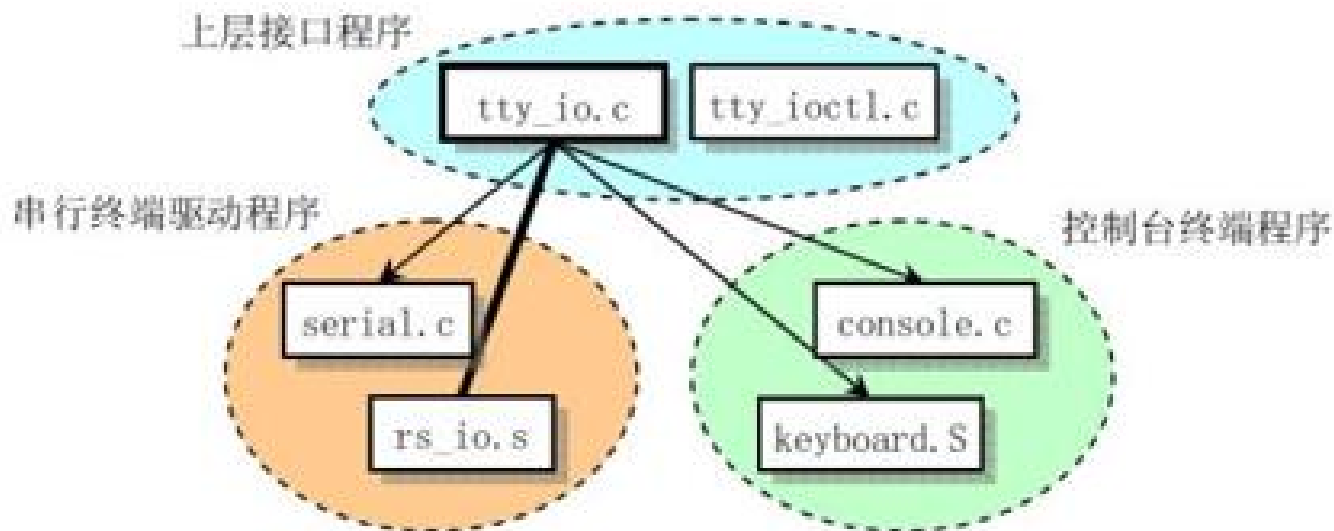
Linux系统中的设备文件

Linux将所有外部设备看成是一类特殊文件，称之为“设备文件”，如果说系统调用是Linux内核和应用程序之间的接口，那么设备驱动程序则可以看成是Linux内核与外部设备之间的接口。设备驱动程序向应用程序屏蔽了硬件在实现上的细节，使得应用程序可以像操作普通文件一样来操作外部设备。

```

peisongwei@ubuntu:~/oslab$ ls
a.out bochsout.txt dbg-asm files gdb-cmd.txt hdc-0.11.img lab4src.tar.gz linux-0.11.tar.gz run tty_io.c
bochs console.c dbg-c gdb hdc keyboard.S linux-0.11 mount-hdc run gdb
peisongwei@ubuntu:~/oslab$ cd linux-0.11/
peisongwei@ubuntu:~/oslab/linux-0.11$ ls
boot fs include init kernel lib Makefile mm tags tools
peisongwei@ubuntu:~/oslab/linux-0.11$ cd kernel/
peisongwei@ubuntu:~/oslab/linux-0.11/kernel$ ls
asm.s chr_drv fork.c math panic.c sched.c sys.c traps.c
blk_drv exit.c Makefile mktime.c printk.c signal.c system_call.s vsprintf.c
peisongwei@ubuntu:~/oslab/linux-0.11/kernel$ cd chr_drv/
peisongwei@ubuntu:~/oslab/linux-0.11/kernel/chr_drv$ ls
console.c keyboard.S Makefile rs_io.s serial.c tty_io.c tty_ioctl.c

```



字符设备程序之间关系示意图

tty_io.c 程序中包含tty字符设备读函数**tty_read()**函数和写函数**tty_write()**,为文件系统提供了上层访问接口。

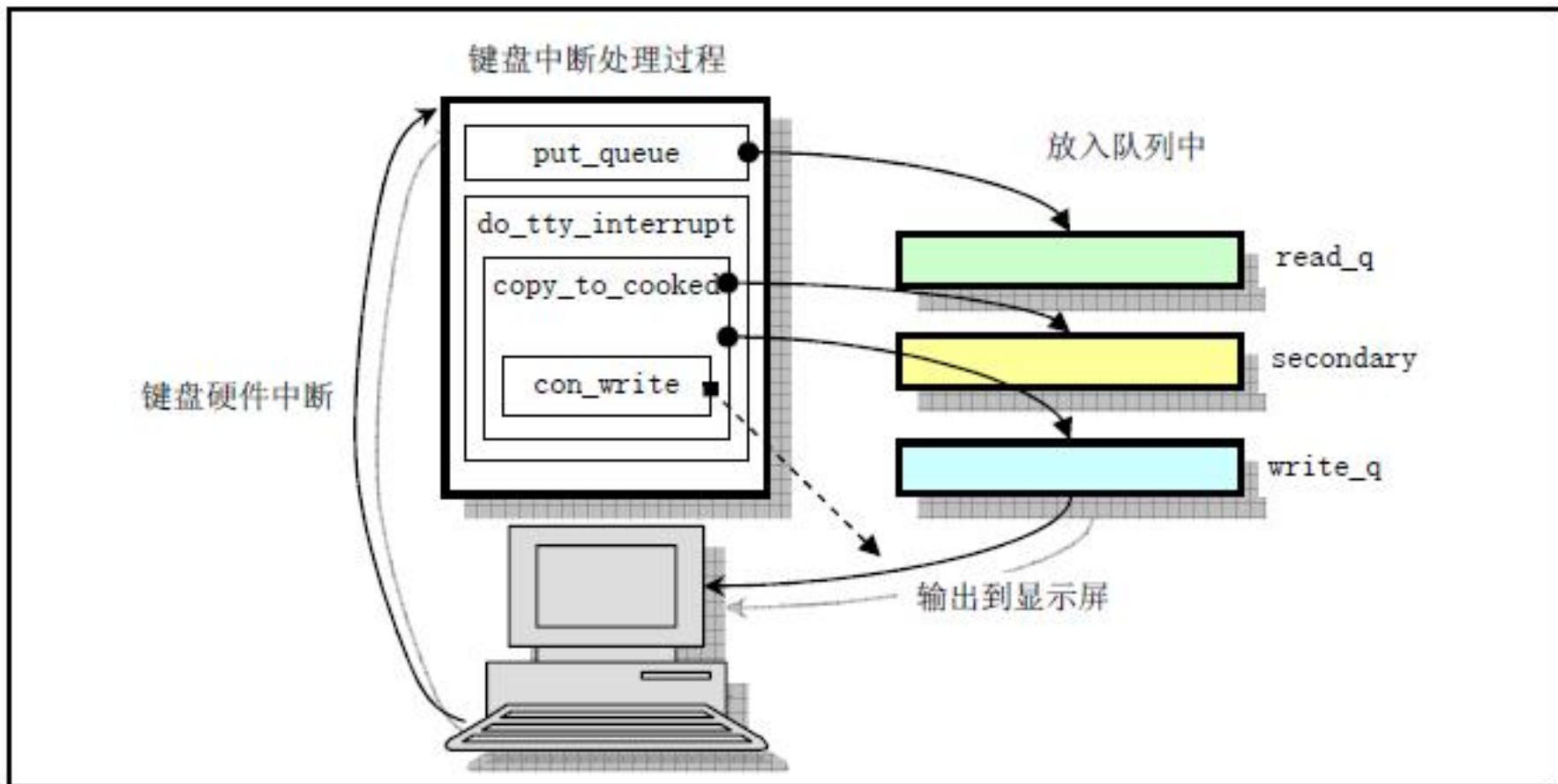
console.c文件主要包含控制台初始化程序和控制台写函数**con_write()**。

rs_io.s 汇编程序用于实现两个串行接口的中断处理程序。

serial.c 用于对异步串行通信芯片UART进行初始化操作。

keyboard.s程序主要实现了键盘中断处理过程
keyboard_interrupt。

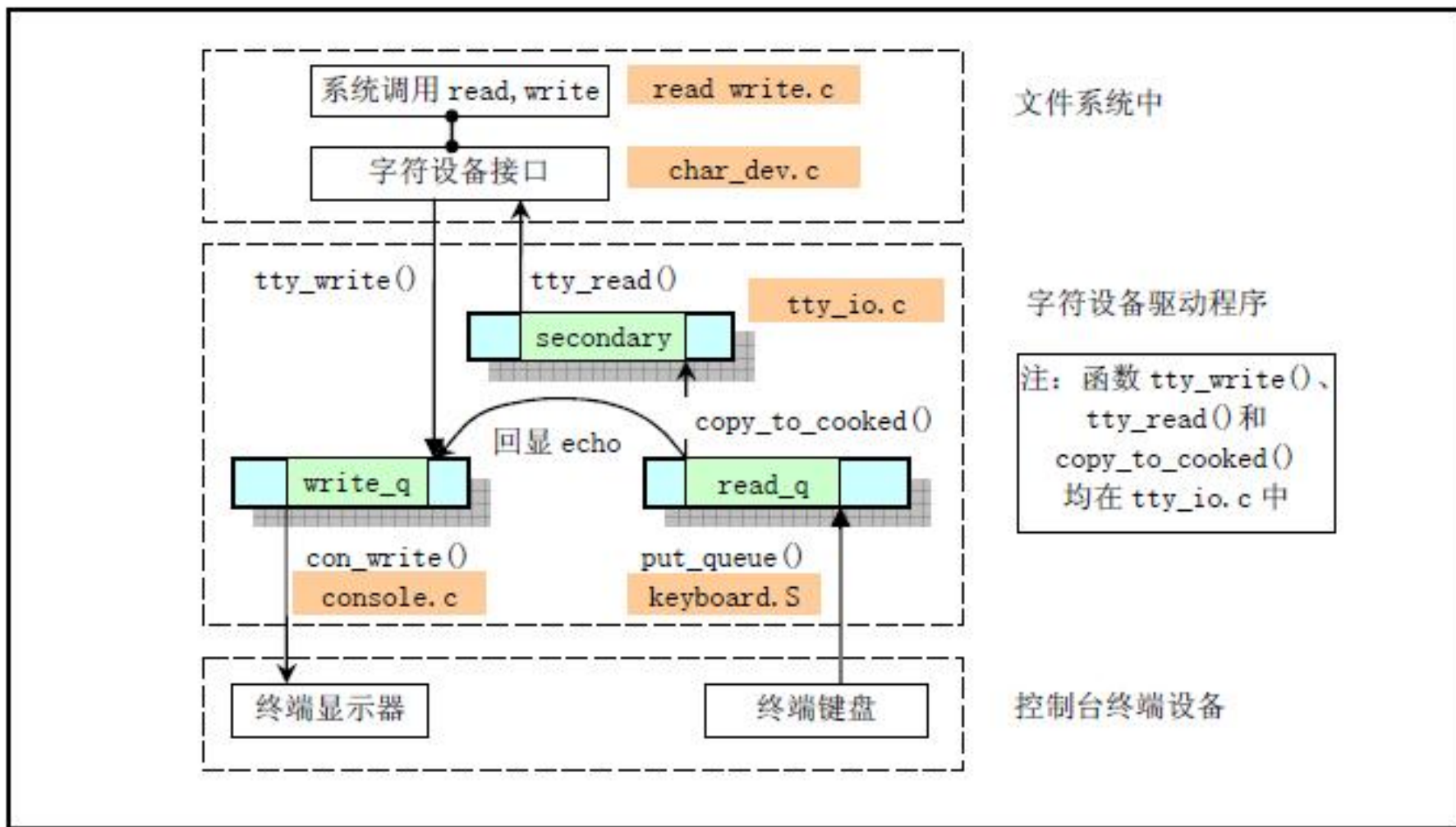
tty_ioctl.c程序实现了tty的IO控制接口函数**tty_ioctl()**



控制台键盘中断处理程序

F1	F2	`	1	2	3	4	5	6	7	8	9	0	-	=	\	BS	ESC	NUML	SCRL	YSR
3B	3C	29	02	03	04	05	06	07	08	09	0A	0B	0C	0D	2B	0E	01	45	46	**
F3	F4	TAB	Q	W	E	R	T	Y	U	I	O	P	[	]			Home	↑	PgUp	PrtSc
3D	3E	0F	10	11	12	13	14	15	16	17	18	19	1A	1B			47	48	49	37
F5	F6	CNTL	A	S	D	F	G	H	J	K	L	;	'	ENTER			←	5	→	-
3F	40	1D	1E	1F	20	21	22	23	24	25	26	27	28	1C			4B	4C	4D	4A
F7	F8	LSHFT	Z	X	C	V	B	N	M	,	.	/		RSHFT			End	↓	PgDn	+
41	42	2A		2C	2D	2E	2F	30	31	32	33	34	35	36			4F	50	51	4E
F9	F10	ALT		Space											CAPLOCK		Ins		De l	
43	44	38		39											3A		52		53	

XT键盘扫描码表



控制台终端字符缓冲队列以及 函数和程序之间的关系

演示实验功能演示：

通过修改驱动程序：当键盘按F12健，字母显示变为*，再按F12，复原。

参考书目：

Linux内核完全注释（电子版）：第10章字符设备驱动（P417）

代码修改：

1、一个全局变量f12Flag，每按一次F12，就将f12Flag的值改变

2、在/kernel/chr_drv/keyboard.s查key_table，可知F1~F12的扫描码用func()处理

```
.long alt,do_self,caps,func      /* 38-3B alt sp caps f1 */
.long func,func,func,func       /* 3C-3F f2 f3 f4 f5 */
.long func,func,func,func       /* 40-43 f6 f7 f8 f9 */
.long func,num,scroll,cursor    /* 44-47 f10 num scr home */
.long cursor,cursor,do_self,cursor /* 48-4B up pgup - left */
.long cursor,cursor,do_self,cursor /* 4C-4F n5 right + end */
.long cursor,cursor,cursor,cursor /* 50-53 dn pgdn ins del */
.long none,none,do_self,func    /* 54-57 sysreq ? < f11 */
.long func,none,none,none       /* 58-5B f12 ? ? ? */
```

代码修改：

3、查看keyboard.s/func()

```

subb $18,%al          // 是功能键 F11, F12 吗?
cmpb $10,%al          // 是功能键 F11?
jb end_func           // 不是, 则不处理, 返回。
cmpb $11,%al          // 是功能键 F12?
ja end_func           // 不是, 则不处理, 返回。

```

即在此处跳转至更改 f12Flag 的函数↵

```

    cmpb $11,%al
    ja end_func
    call change_f12Flag    ↵

```

并添加 .globl keyboard_interrupt,f12Flag ↵

将 change_f12Flag()放在 kernel/chr_drv/console.c 中↵

```
extern int f12Flag=0;

void change_f12Flag(void){
    switch(f12Flag){
        case 1:
            f12Flag=0;
            break;
        case 0:
            f12Flag=1;
            break;
    }
}
```

4、若将更改后的字符回显，则更改 kernel/chr_drv/console.c 中的 con_write() 即可

函数 con_write() 会从终端 tty_struct 结构的写缓冲队列 write_q 中取出字符或字符序列，然后根据字符的性质（是普通字符还是转义字符序列），把字符显示在终端屏幕上或进行一些光标移动、字符擦除等屏幕控制操作。

// 将字符 c 写到显示内存中 pos 处，并将光标右移 1 列，同时也将 pos 对应地移动 2 个字节。

```
__asm__( "movb _attr, %%ah\n\t"
        "movw %%ax, %1\n\t"
        :: "a" (c), "m" (*(short *)pos)
        : "ax");
pos += 2;
```

5、即将字符c在写到内存pos处前，将其更改为*即可即可

```
//if(f12Flag && ((c>64&&c<91)|| (c>96&&c<123)))  
if(f12Flag==1)  
    c = '*';  
  
__asm__ ("movb attr,%%ah\n\t"  
        "movw %%ax,%1\n\t"  
        :: "a" (c), "m" (*(short *)pos)  
        );  
pos += 2;
```

6、若要将更改后的字符写入文件，则要更改
 kernel/chr_drv/tty_io.c中的copy_to_cooked()
 由它来决定是把字符回显,还是把字符放入辅助队列tty->secondary中的，而这个辅助队列中的东西是会写入文件中的

```

    tty->secondary.data++,
    if (L_ECHO(tty)) {
        if (c==10) {
            PUTCH(10,tty->write_q);
            PUTCH(13,tty->write_q);
        } else if (c<32) {
            if (L_ECHOCTL(tty)) {
                PUTCH('^',tty->write_q);
                PUTCH(c+64,tty->write_q);
            }
        } else {
            //if(f12Flag&&((c>64&&c<91)|| (c>96&&c,123)))
            //c='*';
            PUTCH(c,tty->write_q);
        }

        tty->write(tty);
    }
    //write * to files!!!!!!!
    if(f12Flag&&((c>64&&c<91)|| (c>96&&c,123)))
        c='*';
    PUTCH(c,tty->secondary);

```



```
// 如果本地模式标志集中回显标志 ECHO 置位，那么，如果字符是换行符 NL(10)，则将换行符 NL(10)
// 和回车符 CR(13)放入 tty 写队列缓冲区中；如果字符是控制字符(字符值<32)并且回显控制字符标志
// ECHOCTL 置位，则将字符``和字符 c+64 放入 tty 写队列中(也即会显示`C、`H 等)；否则将该字符
// 直接放入 tty 写缓冲队列中。最后调用该 tty 的写操作函数。
    if (L_ECHO(tty)) {
```

7、L_ECHO(tty)控制回显， 若不回显，则在写入缓冲队列前将c更改为*，若回显，在L_ECHO(tty)中将c更改为*为必需，按照第五步输出前更改即可

操作步骤：

安装 VMware Workstation 15 Player↵

1、勾选↵

☐ 增强型键盘驱动程序(需要重新引导以使用此功能(E)
此功能要求主机驱动器上具有 10MB 空间。

2、提示软件更新：安装 VMware Tools for Linux↵

↵

安装 32 位 Linux-Ubuntu 16.04.1 (Ubuntu 16.04.1-desktop-i386.iso , 下载地址：下载地址：<http://old-releases.ubuntu.com/releases/xenial/>) , 其他版本可能会出现 linux 与宿主机间无法复制粘贴文件的现象↵

操作步驟：

安裝gcc-3.4

```
wget http://old-releases.ubuntu.com/ubuntu/pool/universe/g/gcc-3.4/gcc-3.4-base_3.4.6-6ubuntu3_i386.deb  
sudo dpkg -i gcc-3.4-base_3.4.6-6ubuntu3_i386.deb
```

```
wget http://old-releases.ubuntu.com/ubuntu/pool/universe/g/gcc-3.4/cpp-3.4_3.4.6-6ubuntu3_i386.deb  
sudo dpkg -i cpp-3.4_3.4.6-6ubuntu3_i386.deb
```

```
wget http://old-releases.ubuntu.com/ubuntu/pool/universe/g/gcc-3.4/gcc-3.4_3.4.6-6ubuntu3_i386.deb  
sudo dpkg -i gcc-3.4_3.4.6-6ubuntu3_i386.deb
```

```
wget http://old-  
releases.ubuntu.com/ubuntu/pool/universe/g/gcc-  
3.4/libstdc++6-dev_3.4.6-6ubuntu3_i386.deb  
sudo dpkg --force-depends -i libstdc++6-dev_3.4.6-  
6ubuntu3_i386.deb  
wget http://old-  
releases.ubuntu.com/ubuntu/pool/universe/g/gcc-3.4/g++-  
3.4_3.4.6-6ubuntu3_i386.deb  
sudo dpkg -i g++-3.4_3.4.6-6ubuntu3_i386.deb
```

拷贝文件：

```
beisongwei@ubuntu:~$ cd oslab
beisongwei@ubuntu:~/oslab$ ls
a.out  bochsout.txt  dbg-c  gdb  hdc  linux-0.11  mount-hdc  run gdb
bochs  dbg-asm      files  gdb-cmd.txt  hdc-0.11.img  linux-0.11.tar.gz  run
beisongwei@ubuntu:~/oslab$
```

***解压linux-0.11.tar.gz到linux-0.11目录，执行make all,重新make前先执行make clean**

***进入oslab 目录执行./run ,按键观察效果**

***修改源文件keyboard.s;console.c;tty_io.c，再次编译，然后进入oslab目录执行./run，根据实验目的观察实验效果。**

注：1、 ./run init 可把修改文件回复初始状态

2、 前述执行make的过程中，如果提示as86。。。错误信息，请先输入as86按提示信息安装工具

实验任务：

任务1

1) 键入F12，激活*功能，键入学生姓名，将名字首末字母以及名字中包含首末字幕单词修改为*

比如：zhangsan，显示为：*ha*gsa*

2) 再次键入F12，取消该功能，正常显示。

任务2

1) 键入“学号”，激活*功能，键入学生姓名，将名字首末字母以及名字中包含首末字幕单词修改为*

2) 键入“学号-”，取消该功能，正常显示。